

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

Search the Help Center 

[Fireblocks Help Center](#)

[Backup & Recovery](#)

**Native workspace key backup solution**

# Using the Recovery Tool for raw signing

The Raw Signing feature in the Fireblocks Recovery Tool lets you securely sign raw messages or transaction hashes with your recovered private keys without ever exposing your keys to an online environment.

This feature provides flexibility for advanced recovery scenarios, such as manually broadcasting transactions or interacting with smart contracts outside the usual Fireblocks workflow.

## Before you begin

- Recover your private keys using the **Utility app**.
- Choose whether you want to:
  - Generate a signature for a hashed message, or-
  - Sign a QR code generated by the Relay app

---

## Option 1: Generate a signature

1. In the Recovery Tool, select a wallet from your restored vault accounts or select a derivation path and signing algorithm (ECDSA or EDDSA).
2. Paste the message or transaction hash you want to sign.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

1f87bd6725de122005

3. Select **Sign Message**. A signature will appear in a modal window.

- **EDDSA (e.g. Solana)**: Paste your Solana message into the tool and generate a signature. Import the signature into an online machine, add it to the transaction, and broadcast to the blockchain.
- **ECDSA (e.g. Ethereum)**: Paste the Ethereum transaction hash, generate the (r+s+v) signature, and apply it to your unsigned transaction before broadcasting.

Raw Signing

ECDSA Signature Components:

Full Signature:  
cfc6d57c087e32d78a0cbe471d1a0b861cbe20965a998161513ee86a535973af77ef89402d21246cf6da72c839914bd61d2bdb1e57802e96b71c5afbc5285b1d1c

r:  
0xcfc6d57c087e32d78a0cbe471d1a0b861cbe20965a998161513ee86a535973af

s:  
0x77ef89402d21246cf6da72c839914bd61d2bdb1e57802e96b71c5afbc5285b1d

v:  
28

## Option 2: Sign via QR code

1. In the **Relay app**, go to **Raw Sign**.
2. Select a wallet or derivation path and enter the message you want to sign.
3. A QR code will be generated.

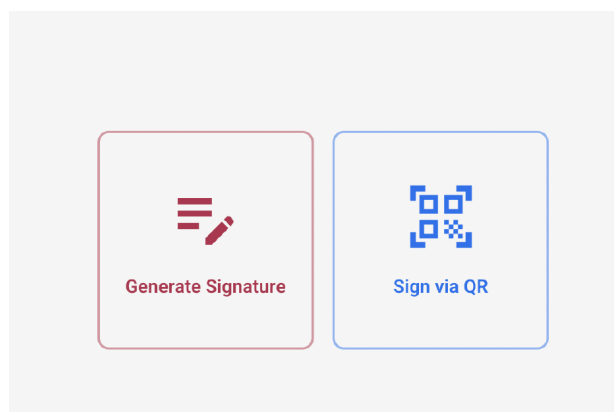
Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



4. In the **Utility app** (offline), select **Sign via QR** and scan the QR code. Your private key stays secure and never leaves the offline device.

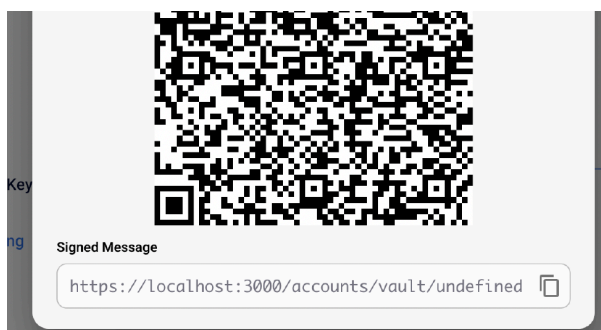


5. A new QR code containing the signature is then generated.

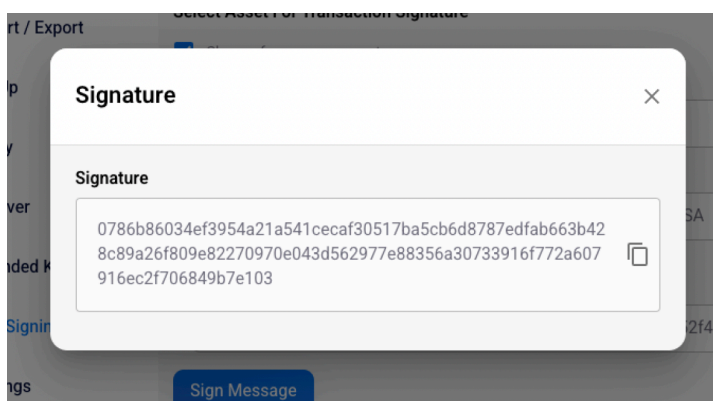
Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



6. Scan this signature QR code back in the Relay app to complete the process.
7. The signed message will appear, ready for broadcasting.



## Using unsigned messages

If you need to prepare unsigned transactions for signing:

- For Solana, you can create an unsigned transfer transaction (or other Solana transactions) using the Solana SDK. Once done, sign it offline with the Recovery Tool and broadcast it once signed.
- For **Ethereum**, you can prepare role-change or contract transactions. Use the Recovery Tool to sign the transaction hash and then attach the r+s+v signature components before broadcasting.

## Code examples

### Solana

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

## Install the necessary SDK package

```
npm install @solana/web3.js
```

## Generate an unsigned message

```
import * as web3 from "@solana/web3.js";
const generateMessage = async () => {
  const connection = new web3.Connection(
    web3.clusterApiUrl("mainnet-beta"),
    "confirmed"
  );
  const tx = new web3.Transaction();
  const from = new web3.PublicKey(
    "H682Lm8MgXVNz5u9hP7Wh6JM257uKd5sn3CGiHa7eJBD" // the solana wallet
  );
  tx.add(
    web3.SystemProgram.transfer({
      fromPubkey: from,
      toPubkey: new web3.PublicKey(
        "5Q1CJ9ERHt4hj3bSymkg8Dd924891Ky4mRDyUENVddE9" // the solana address
      ),
      lamports: web3.LAMPORTS_PER_SOL, // 1 SOL
    })
  );
  const recentBlockhash = await connection.getLatestBlockhash();
  tx.recentBlockhash = recentBlockhash.blockhash;
  tx.feePayer = from;
  const messageToSign = tx.serializeMessage();
  console.log("Message to sign:", messageToSign.toString("hex"));
  console.log(
    "Transaction: ",
  );
}
```

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

```
return {  
  transaction: tx,  
  connection: connection,  
  serializedTx: tx.serialize({ verifySignatures: false }).toString("  
};  
};
```

### Add a signature to the message

```
const addSignature = async (  
  tx: String,  
  signatureHex: string,  
  signerAddress: string  
) => {  
  const newTx = web3.Transaction.from(Buffer.from(tx, "hex"));  
  const pubkey = new web3.PublicKey(signerAddress);  
  console.log("New Transaction: ", newTx.serializeMessage().toString("  
  newTx.addSignature(pubkey, Buffer.from(signatureHex, "hex"));  
  console.log("Verified Transaction: ", newTx.verifySignatures(true));  
  return newTx;  
};
```

### Broadcast the message to the blockchain

```
const broadcastTransaction = async (  
  connectionUrl: string,  
  signedTransactionHex: string  
) => {  
  try {  
    const connection = new web3.Connection(connectionUrl, "confirmed")  
    const signedTransaction = web3.Transaction.from(  
      Buffer.from(signedTransactionHex, "hex")  
    );
```

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

```
        skipPreflight: false, // Set to true to skip preflight checks
        preflightCommitment: "confirmed",
        maxRetries: 3,
    }
);
console.log("Transaction sent! Signature:", signature);
console.log(
    `View on Solana Explorer: https://explorer.solana.com/tx/${signature}`
);
// Wait for confirmation
console.log("Waiting for confirmation...");
const confirmation = await connection.confirmTransaction({
    signature: signature,
    blockhash: signedTransaction.recentBlockhash!,
    lastValidBlockHeight: (
        await connection.getLatestBlockhash()
    ).lastValidBlockHeight,
});
if (confirmation.value.err) {
    throw new Error(
        `Transaction failed: ${JSON.stringify(confirmation.value.err)}`
    );
}
console.log("Transaction confirmed!");
return {
    signature: signature,
    confirmed: true,
    explorerUrl: `https://explorer.solana.com/tx/${signature}`,
    confirmation: confirmation,
};
} catch (error) {
    console.error("Error broadcasting transaction:", error);
    throw error;
}
};
```

## Ethereum

△ Fireblocks Fireblocks Help C

7

Below is a list of common role IDs used in the ERC20F contract:

- This example guides you through preparing a raw transaction for a role change, generating the hash for offline signing, and then using the Recovery Tool to sign and broadcast the transaction securely.

- Run the code example as explained in the [Github repository instructions](#) file:

```
> grant-role-offline@1.0.0 start
> npm run build && node dist/changeRoleRemotely.js

> grant-role-offline@1.0.0 build
> tsc

? Enter the contract address: 0x52272d21B1aE751200ECC23eFe21F4BC003e5221
```



Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

```
grant-role-offline@1.0.0 build
> tsc

? Enter the contract address: 0x52272d21B1aE751200ECC23eFe21F4BC003e522F
? Enter the sender (from) address: 0x2Ffefa22A88Bf5eDaCE966133908a66220876e8f
```

## Enter the recipient's address

```
> grant-role-offline@1.0.0 build
> tsc

? Enter the contract address: 0x52272d21B1aE751200ECC23eFe21F4BC003e522F
? Enter the sender (from) address: 0x2Ffefa22A88Bf5eDaCE966133908a66220876e8f
? Enter the account to grant the role to (to address): 0xf1B81a5698e3E17b798B85F220dDfecBb241E8F2
```

## Select the role to grant

```
? Enter the contract address: 0x52272d21B1aE751200ECC23eFe21F4BC003e522F
? Enter the sender (from) address: 0x2Ffefa22A88Bf5eDaCE966133908a66220876e8f
? Enter the account to grant the role to (to address): 0xf1B81a5698e3E17b798B85F220dDfecBb241E8F2
? Select the role to grant: (Use arrow keys)
> DEFAULT_ADMIN_ROLE (0x0000000000000000000000000000000000000000000000000000000000000000)
MINTER_ROLE (0x9f2df0fed2c77648de5860a4cc508cd0818c85b8b8a1ab4ceef8d981c8956a6)
BURNER_ROLE (0x3c11d16cbaffd01df69ce1c404f6340ee057498f5f00246190ea54220576a848)
PAUSER_ROLE (0x65d7a28e3265b37a6474929f336521b332c1681b933f6cb9f3376673440d862a)
RECOVERY_ROLE (0x0acf805600123ef007091da3b3ffb39474074c656c127aa68cb0ffec232a8ff8)
```

## Get the hash from the console and paste it into the recovery tool

Please sign this hash with your external tool, then provide the signature components below:

```
? Enter signature r value (with 0x prefix):
0xcfc6d57c087e32d78a0cbe471d1a0b861cbe20965a998161513ee86a535973af
? Enter signature s value (with 0x prefix):
0x77ef89402d21246cf6da72c839914bd61d2bdb1e57802e96b71c5afbc5285b1d
? Select v value: 28
? Confirm signature details and proceed with transaction? (Y/n) Y
```

## Add signature from the Recovery Utility, serialize the transaction, and broadcast it

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

View on Etherscan: <https://etherscan.io/tx/0xf39a086be8b62cc7908220d5d5a69ae2add36fd82013f3b0b3c9a5b32d9b429>

Waiting for confirmation...

Was this article helpful?

Yes

No

#### ON THIS PAGE

##### Before you begin

Option 1: Generate a signature

Option 2: Sign via QR code

Using unsigned messages

Code examples

#### Related Articles

-  Raw Signing
-  Transactions not broadcast by Fireblocks
-  Typed Message Signing
-  Using dApps on non-EVM networks
-  Managing your Movement account on Fireblocks
-  Staking with Raw Signing

Don't see what you're looking for? Log in for the full Help Center experience



Fireblocks Help Center



---

Fireblocks © 2026. All Rights Reserved. NMLS Registration Number: 2066055

[Privacy Policy](#)